

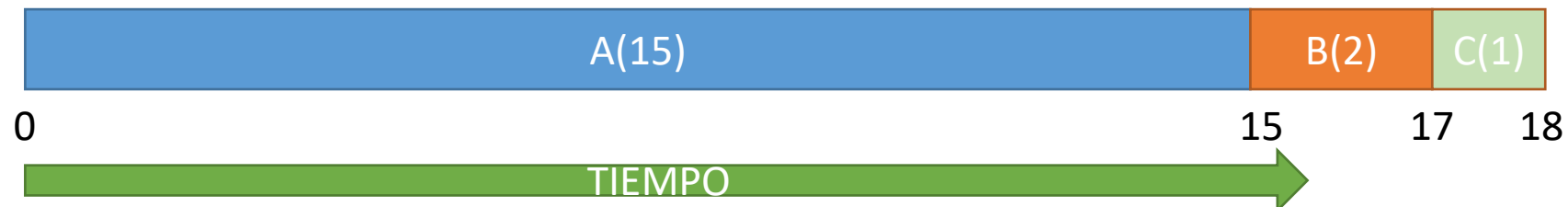
Algoritmos de Asignación de Procesos

First Come/First Served (FCFS) (1)

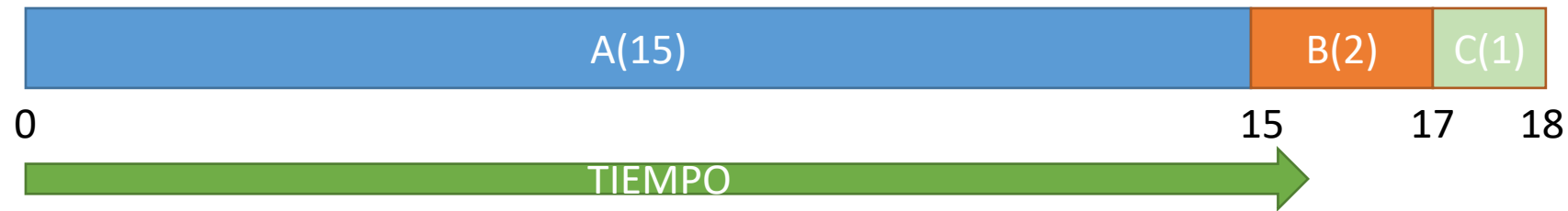
- En estos sistemas, se procede con filosofía FIFO, por lo que apenas se crea la PCB se le coloca al final de una cola de de Procesos.
- En estos sistemas no hay status WAIT: cada proceso es ejecutado hasta su completitud.
- Vamos a trabajar a partir de ahora, en cada escenario con procesos contados por cantidad de milisegundos para compeltarse.

First Come/First Served (2)

- Supongamos que tenemos 3 procesos LISTOS (READY) con la siguiente duración de ciclos de CPU:
 - A, dura 15 milisegundos
 - B dura 2 milisegundos
 - C dura 1 milisegundo.
- Cada caso considera uso del CPU y requests de entrada salida.
- En un modelo FC/FS si lo importante ES EL ORDEN, si llegan A,B,C la ejecución sería:



First Come/First Served (3)



El concepto clave para valorar la estrategia a seguir es el TURNAROUND TIME= el *TIEMPO REQUERIDO para completar un proceso o solicitud desde el momento en que está LISTO.*

En este caso, los procesos duran, desde que están listos:

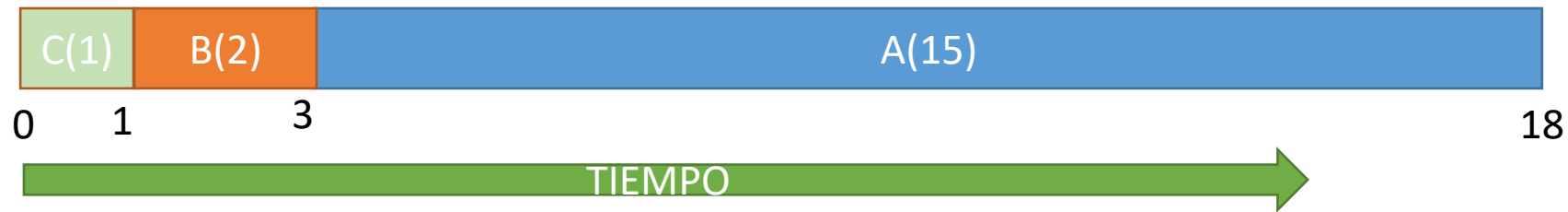
A = 15ms

B = 17ms

C = 18ms.

El tiempo promedio de TURNAROUND TIME es: $(15+17+18)/3 = 16.67$

First Come/First Served (4)



Si hubieran llegado en OTRO ORDEN, C, B, A.....

El tiempo promedio de TURNAROUND TIME es: $(1+3+18)/3 = 7.3$

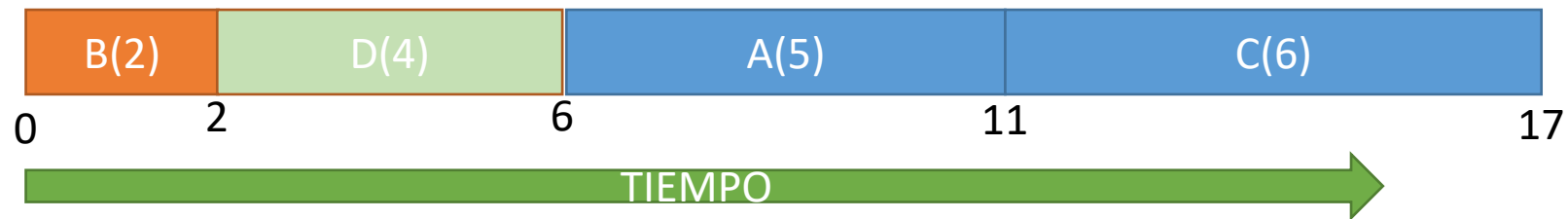
Está claro que aunque es muy sencillo de implementar, este método tiene una gran debilidad: el tiempo promedio varia demasiado y una gran eficiencia depende de la “suerte” en el orden en que llegan los procesos.

Shortest Next Job (ó Job Next) (1)

- La ejecución se basa en la duración del ciclo de CPU para ejecutar un trabajo o proceso.
- Supongamos que tenemos 4 procesos LISTOS (READY) con la siguiente duración de ciclos de CPU:
 - A, 5 milisegundos
 - B 2 milisegundos
 - C 6 milisegundos
 - D 4 milisegundos.
- De nuevo, cada caso considera uso del CPU y requests de entrada salida.

Shortest Next Job (2)

El Process Scheduler revisa el Código y “estima” la duración. Los ejecutará en el siguiente orden



El tiempo promedio de TURNAROUND TIME es: $(2+6+11+17)/4 = 9.0$

Este algoritmo es muy fácil de implementar en SO de “batch” o lotes de trabajo, pero en sistemas interactivos es prácticamente imposible.

Es decir, si sé el volúmen de datos de cada proceso y cuánto tiempo dura el proceso total, es útil.

Su debilidad radica además en que la “estimación” debe ser lo más exacta posible

Prioridad (1)

- Este algoritmo es el más común en sistemas de batch porque da preferencia a procesos importantes.
- Basicamente funciona ordenando la cola de procesos con PCB creados según la prioridad. (Si 2 procesos tienen la misma prioridad, se usa criterio FIFO).
- Los criterios pueden ser:
 - Acceso a memoria
 - Acceso a dispositivos E/S
 - Tiempo de operaciones de CPU
 - Tiempo que el proceso YA LLEVA en el Sistema (aging).

Próximo a terminar (1)

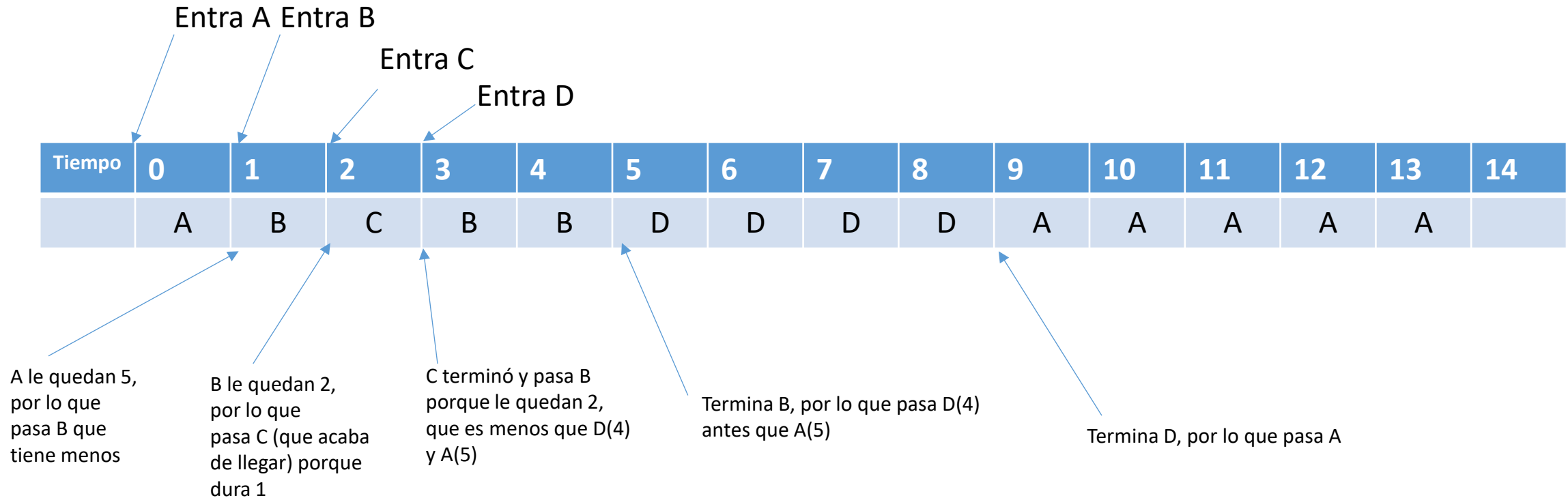
- Este algoritmo es una variante del “Proceso más corto” (Shortest Next Job) donde el algoritmo escoge el proceso que está “más cerca” de terminar.
 - El criterio más básico es que tan cerca está el puntero de instrucciones del final de las instrucciones del proceso.
- Supongamos que tenemos 4 procesos LISTOS (READY) con la siguiente duración de ciclos de CPU:
 - A, dura 6ms
 - B, dura 3ms
 - C, dura 1ms
 - D, dura 4ms

Proximo a terminar (2)

- PERO!! Si hay un proceso EN LA COLA que dura menos, se le da prioridad!!
- Supongamos entonces que ADEMÁS llegan en este orden

MS de llegada	Proceso	Duracion
0	A	6
1	B	3
2	C	1
3	D	4

Próximo a terminar



Aunque parece confuso, en este algoritmo el “Turnaround time” es TIEMPO DE TERMINACIÓN – TIEMPO DE ENTRADA

$$A = 14 - 0 = 14$$

$$B = 5 - 1 = 4$$

$$C = 3 - 2 = 1$$

$$D = 9 - 3 = 6$$

$$\text{Turnaround Time} = (14 + 4 + 1 + 6) / 4 = 6.25$$

Round Robin

- Este es el Sistema de trabajo más común y usado en los Sistemas Operativos INTERACTIVOS.
- El CPU se asigna por medio de dividir su tiempo en segmentos fijos de tiempo llamados QUANTUMS (time quantum)
 - Los procesos pasan usando filosofía FIFO.
 - Si el proceso termina antes de que su quantum acabe, se pasa al siguiente proceso.
 - Si el proceso es interrumpido (INTERRUPCIÓN) entonces se procede como ya conocemos (salva el estado del procesador, ejecutar el manejador de Interrupciones correspondiente y restaurar el estado).

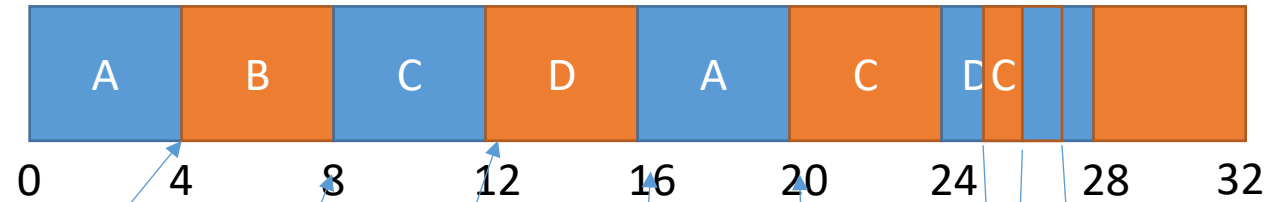
Round Robin

MS de llegada	Proceso	Duracion
0	A	8ms
1	B	4ms
2	C	9ms
3	D	5ms

Supongamos el siguiente escenario de procesos con tiempos de arribo y duración (Ignoramos interrupciones).

¿Cómo se vería la ejecución si suponemos Quantum de 4ms?

Round Robin



Sale A, le quedan 4

Sale C, le quedan 5

Sale D, le queda 1

Sale B, ya se completó

A termina... Vuelve C y luego D y al cierre C de nuevo

¿Qué hacen los SO más modernos?

- Los sistemas operativos hoy día son en su mayoría interactivos.
- Sistemas Operativos como Linux, Windows o MacOS usan ROUND ROBIN y lo complementan con algunas de las técnicas vistas aquí para acelerar algunas tareas, especialmente se usan técnicas de colas de prioridad.

Ejercicio

- Dada la siguiente tabla:

Job	Tiempo de Arribo	Duración en Ciclos
A	0	2
B	1	12
C	2	4
D	4	3
E	5	8
F	7	5
G	8	3

1. Calcule el turnaround time usando First come, First serve con el orden A,B,C,...
2. Calcule el turnaround time usando shortest next job con el orden dado (a,b,c..)
3. Calcule el turnaround time usando proximo a terminar usando la información dada
4. Calcule el turnaround time usando asignación por prioridad asumiendo que todos tienen prioridad 0 (básica), pero C, E y G tienen prioridad mayor (1,2 y 3 respectivamente)
5. Calcule el turnaround time usando Round Robin con quantums de 4 ciclos y 5 ciclos. CAMBIE PROCESO CON EL QUANTUM
6. Calcule el turnaround time usando Round Robin con quantums de 5 y 6 ciclos. NO CAMBIE PROCESO AL CAMBIAR QUANTUM SI EL PROCESO SOLO EJECUTÓ 2 O MENOS CICLOS. Compare resultados con el punto 5